

Simulation Without a Black Box

Outline

Why Extremes Reveal Bugs

Empty System Test

Overloaded System Test

Single-Customer Trace

Degenerate Parameter Tests

Inventory Model Boundaries

pytest Parametric Fixtures

Key Takeaways

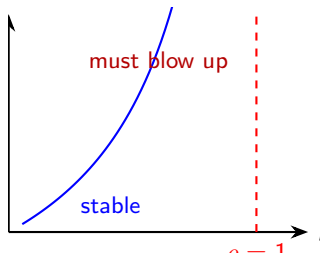
Why Extreme Conditions Reveal Bugs

Boundary Behavior Principle

Bugs hide in normal operating ranges but are exposed at extremes because every branch of logic is forced to execute or is intentionally bypassed.

Three boundary classes:

1. **Empty system** — no load at all
2. **Overloaded system** — demand exceeds capacity
3. **Degenerate parameters** — service time $\rightarrow 0$, infinite or zero patience, zero lead time



Empty System Test: Zero Arrivals

Setup: $\lambda \approx 0$, run for $T = 10\,000$.

Expected results:

- Utilisation $\rho = 0$
- Mean queue length $L_q = 0$
- Zero customers served
- No ZeroDivisionError

Common Failure

Dividing by `num_served` when it equals zero raises a runtime error. Guard every KPI denominator with a conditional.

```
def test_zero_arrival():  
    q = MM1Queue(  
        arrival_rate=1e-9,  
        service_rate=1.0,  
        seed=0)  
    r = q.run(sim_time=10_000)  
    assert r["utilisation"] < 1e-4  
    assert r["Lq"] < 1e-4  
    assert r["num_served"] == 0
```

Overloaded System: $\rho > 1$

Setup: $\lambda = 1.5$, $\mu = 1.0$.

Expected behavior:

- Queue grows *linearly* in time
- $L_q(T) \approx (\lambda - \mu) T/2$
- Throughput $\approx \mu$ (server permanently busy)

Instability Diagnostic

Plot $L_q(t)$ vs. t . Stable $\Rightarrow$ flat trend.

Unstable $\Rightarrow$ rising slope. If overloaded model shows a flat trend, the entity-counting logic is broken.

```
def test_overload_grows():  
    q = MM1Queue(  
        arrival_rate=1.5,  
        service_rate=1.0,  
        seed=7)  
    r = q.run(sim_time=5_000,  
              track_queue=True)  
    # final queue >> early queue  
    assert (r["Lq_final"]  
            > 10 * r["Lq_t100"])
```

Warning

Never report steady-state KPIs from an unstable simulation — results are mathematically meaningless.

Single-Customer Trace: Manual Verification

Setup: One customer, deterministic inter-arrival = 0, deterministic service time = $s = 2$.

Hand-calculated events:

1. $t = 0$: arrives, server idle $\Rightarrow$ starts immediately, $W_q = 0$
2. $t = 2$: departs; $W = 2.0$
3. $t > 2$: server idle, $\rho = 2/T$

Simulation output must match exactly.

Why This Matters

If a single deterministic customer fails the hand trace, every result from that model is wrong. This is the cheapest possible bug detector.

```
q = MM1Queue(
    arrival_rate=1e-9,
    service_rate=0.5, # s=2
    seed=0)
r = q.run(sim_time=100)
assert r["Wq"] < 1e-6
assert abs(r["W"] - 2.0) < 1e-6
```

Degenerate Parameter Test Table

Condition	Expected limit	Bug signal
Service time $\rightarrow 0$	$W \approx 1/\lambda$, $W_q \approx 0$	$W_q < 0$ (clock error)
$\lambda \rightarrow \mu^-$	$W_q \rightarrow +\infty$	Finite, bounded W_q
Patience $\rightarrow \infty$	M/M/1/K as $K \rightarrow \infty$	Entity count mismatch
Patience = 0	All renege; $W_q = 0$	Non-zero W_q reported
Lead time = 0	Stock replenished instantly	Stockouts still occur
Demand rate = 0	Stock never depletes; no orders	Spurious orders placed

Warning

Degenerate limits are *not* edge cases to ignore — they are the cheapest possible regression tests. Add them to your pytest suite from day one.

Inventory Model: Boundary Tests

(s, S) inventory — three boundaries:

1. **Demand** = 0: stock stays at S ; no order ever placed.
2. **Demand** $\rightarrow \infty$: perpetual stockout; fill rate $\rightarrow 0$.
3. **Lead time** = 0: order arrives instantly; stock never drops below reorder point s .

```
def test_zero_demand():
    inv = SSInventory(
        s=10, S=50,
        demand_rate=0.0, seed=0)
    r = inv.run(sim_time=1_000)
    assert r["stockouts"] == 0
    assert r["num_orders"] == 0

def test_instant_replenishment():
    inv = SSInventory(
        s=10, S=50,
        demand_rate=2.0,
        lead_time=0.0, seed=1)
    r = inv.run(sim_time=5_000)
    assert r["min_stock"] >= 10
```

pytest Fixtures for Parametric Boundary Tests

```
import pytest
from simdes.models.queues import MM1Queue

@pytest.fixture(params=[0.1, 0.5, 0.7, 0.9])
def rho(request):
    return request.param

def test_utilisation_matches_rho(rho):
    mu = 1.0
    q = MM1Queue(arrival_rate=rho * mu, service_rate=mu, seed=42)
    r = q.run(sim_time=200_000)
    assert abs(r["utilisation"] - rho) < 0.02

def test_wq_formula(rho):
    mu = 1.0; lam = rho * mu
    wq_theory = lam / (mu * (mu - lam))
    q = MM1Queue(arrival_rate=lam, service_rate=mu, seed=42)
    r = q.run(sim_time=200_000)
    assert abs(r["Wq"] - wq_theory) / wq_theory < 0.05
```

Key Takeaways

L34 Summary

1. Extreme conditions expose bugs that nominal-range tests miss — always test $\lambda = 0$, $\rho \rightarrow 1^-$, and $\rho > 1$.
2. A single deterministic customer *must* reproduce the hand trace exactly before you trust anything else.
3. Service time $\rightarrow 0$ and patience limits are cheap but powerful checks.
4. Inventory models have their own boundary triad: zero demand, instant replenishment, infinite demand.
5. `pytest.fixture(params=...)` automates parametric sweeps with one test function covering all load levels.

Next: L35 — Validation evidence: comparing model output to reality.